CracksLatinoS! 2006

-* Concurso 110 Nivel4 *-

Programa	UnPackMe_VMProtect 1.24
Download	FTP de Ricardo
Descripción	Ejecutable empakado con VMProtect 1.24
Herramientas	OllyDbg 1.10 + ImpREC 1.6f + ToPo 1.2 + RDG Packer v0.6.5 Beta
Dificultad	Burno esto depende de cada uno..
Compresor/Compilador	VMProtect 1.24
Protección	Empaquetado con VMProtect 1.24
Objetivos	Desempacar manualmente
Cracker	SyXe'05 **Fecha:** 19/11/06
Tutorial n°	6° concurso

-** Introduccion **-

Buenas a todos los CracksLatinos, nos volvemos a ver las caras otra vez, jeje.. esta vez para tratar de desempakar un crackme ☺

En realidad vamos a hincarle el diente al Concurso110 Nivel4 y miraremos de solucionarlo, eso sí, el packer es muy puñetero¡!!

-** Explorando el objetivo **-

Bueno aunque nos diga que es un VMProtect 1.24 lo vamos a verificar con los analizadores PEiD 0.94 (autores: snaker, Qwerton y Jibz) y RDGPacker Detector 0.6.5 Beta (autor: rdgmax) a ver qué nos dicen:

Nothing found *

Esto con la opción "Normal Scan" si miro con "Deep Scan" y "Hardcore Scan" me saca esto, jeje..

Microsoft Visual C++ 5.0

O sea que lo ve como si fuera un MSVC++ 5.0 ¿?

Y bueno, en vista del poco éxito que últimamente está teniendo PEiD ya que lamentablemente parece ir en decadencia utilizo el RDGPacker Detector 0.6.5 BETA a ver que encuentra:

Vale este sí atinó como ya nos tiene acostumbrados, pues su autor rdgmax tiene las signaturas muy actualizadas, utilicé el escaneo Externo tipo M-B.

Ahora que confirmamos lo que dice el enunciado nos podemos meter de lleno con el packer a ver que encontramos ☺

-** Desempacando **-

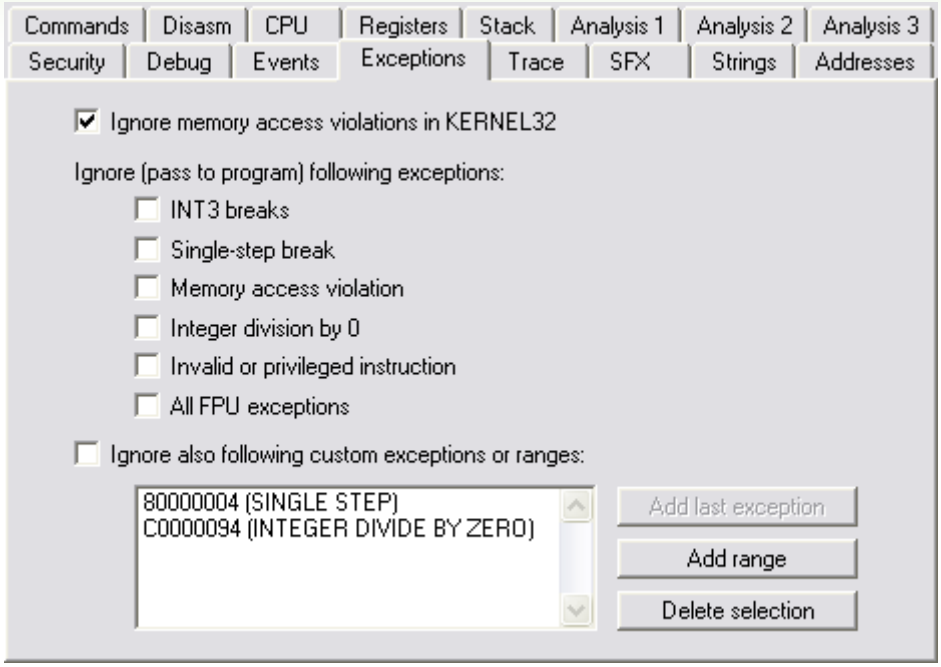
Pues nada, Cargamos el EXE en Olly y para en su EntryPoint (EP):

Address	Hex dump	Disassembly		Comment
004271B0	$ 68 AB9D0C88	push	880C9DAB	EP del packer!!
004271B5	. E8 6A540400	call	0046C624	
004271BA	. 8B45 00	mov	eax,[ebp]	
004271BD	. 36:8B18	mov	ebx,ss:[eax]	
004271C0	. 895D 00	mov	[ebp],ebx	
004271C3	.v E9 9F000000	jmp	00427267	
004271C8	. 66:8B06	mov	ax,[esi]	
004271CB	. 98	cwde		
004271CC	. 83ED 04	sub	ebp,4	
004271CF	. 83EE FE	sub	esi,-2	
004271D2	. 8945 00	mov	[ebp],eax	
004271D5	.- E9 3E430400	jmp	0046B518	

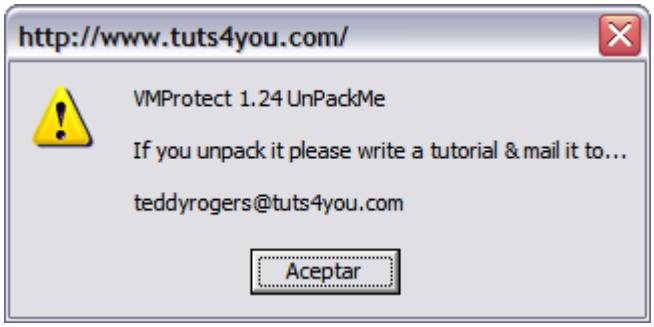
Si nos fijamos vemos que es el OEP del UnPackMe de Teddy Rogers (0x4271B0), o sea que ya nos cambian los esquemas, arranca desde el propio OEP (todavía encriptado), entonces la pregunta obligada es ¿será como Petite? cuyo EP y OEP coinciden en cuanto a dirección, es decir, arranca desde una ubicación X y después X será el OEP.

Veamos si es así, lo más fácil es colocar un HBP en 4271B0 y ejecutar con F9 (RUN) para ver que no es la idea, de todos modos este método jamás podría ser definitivo porque siempre partiremos de un OEP desconocido, en este caso lo sabemos por el unpackme que venimos tratando. Por lo tanto busquemos un método alternativo de descubrir el OEP y que nos pueda funcionar cuando desconozcamos este.

Lo primero que se me viene a la cabeza es el método de Ricardo de las EXCEPCIONES, primero las habilito TODAS quitando el check:



Esto es para que Olly pare al encontrar alguna excepción, y aver si así averiguamos su OEP, damos RUN con F9 y...



El UnPackMe echa a correr, hummmmmmmm... así que NADA de Excepciones, bueno pues nada a tracear que se ha dicho, a ve que hacemos ??

Si traceamos un poco enseguida nos damos cuenta de este salto indirecto:

Address	Hex dump	Disassembly		Comment
00427260	. 89E7	mov	edi,esp	
00427262	> 89F3	mov	ebx,esi	
00427264	. 0375 00	add	esi,[ebp]	
00427267	> 8A1E	mov	bl,[esi]	
00427269	. 83EE FF	sub	esi,-1	
0042726C	. 0FB6C3	movzx	eax,bl	
0042726F	.v FF2485 00B04	jmp	[eax*4+46B000]	UnPackMe.0042728C
00427276	. 0FB606	movzx	eax,byte ptr [esi]	
00427279	. 66:8B4D 00	mov	cx,[ebp]	
0042727D	. 83EE FF	sub	esi,-1	
00427280	. 83C5 02	add	ebp,2	
00427283	. 66:890C07	mov	[edi+eax],cx	
00427287	.^ E9 DBFFFFFF	jmp	00427267	
0042728C	. 80E0 3C	and	al,3C	
0042728F	. 8B4D 00	mov	ecx,[ebp]	
00427292	. 83C5 04	add	ebp,4	
00427295	. 890C07	mov	[edi+eax],ecx	
00427298	.^ E9 CAFFFFFF	jmp	00427267	
0042729D	. 8B4D 00	mov	ecx,[ebp]	
004272A0	. 66:8B5D 04	mov	bx,[ebp+4]	
004272A4	. 83C5 06	add	ebp,6	
004272A7	. 66:8919	mov	[ecx],bx	
004272AA	.^ E9 B8FFFFFF	jmp	00427267	
004272AF	. 8B5D 00	mov	ebx,[ebp]	
004272B2	. 8B4D 04	mov	ecx,[ebp+4]	
004272B5	. 83C5 08	add	ebp,8	
004272B8	. 36:890B	mov	ss:[ebx],ecx	
004272BB	.^ E9 A7FFFFFF	jmp	00427267	
004272C0	. 8B55 00	mov	edx,[ebp]	
004272C3	. 66:8B0A	mov	cx,[edx]	
004272C6	. 83C5 02	add	ebp,2	

Si nos fijamos veremos que está continuamente parando en este salto, y tampoco podemos colocar un BPM en la sección de código porque aquí se está ejecutando el packer y de nada serviría. De momento coloquemos un BP en dicho salto y vayamos dando RUN hasta que cambie de valor:

```
Address   Hex dump        Disassembly                    Comment
00427264  .  0375 00      add     esi,[ebp]
00427267  >  8A1E         mov     bl,[esi]
00427269  .  83EE FF      sub     esi,-1
0042726C  .  0FB6C3       movzx   eax,bl
0042726F  .- FF2485 00B04 jmp     [eax*4+46B000]         UnPackMe.0046B463
00427276  .  0FB606       movzx   eax,byte ptr [esi]
00427279  .  66:8B4D 00   mov     cx,[ebp]
0042727D  .  83EE FF      sub     esi,-1
00427280  .  83C5 02      add     ebp,2
00427283  .  66:890C07    mov     [edi+eax],cx
00427287  .^ E9 DBFFFFFF  jmp     00427267
0042728C  .  80E0 3C      and     al,3C
0042728F  .  8B4D 00      mov     ecx,[ebp]
```

Ahí está ahora saltará hacia :46B463 donde no hay nada importante a la vista:

```
Address   Hex dump        Disassembly                    Comment
0046B463    89E8          mov     eax,ebp
0046B465    83ED 04       sub     ebp,4
0046B468    8945 00       mov     [ebp],eax
0046B46B  v E9 A8000000   jmp     0046B518
0046B470    8A06          mov     al,[esi]
0046B472    46            inc     esi
0046B473    66:8B0407     mov     ax,[edi+eax]
0046B477    83ED 02       sub     ebp,2
0046B47A    66:8945 00    mov     [ebp],ax
0046B47E  v E9 95000000   jmp     0046B518
0046B483    8B45 00       mov     eax,[ebp]
0046B486    66:36:0FB608  movzx   cx,byte ptr ss:[eax]
0046B48B    83C5 02       add     ebp,2
```

Pues NADA sigamos con el BP puesto y dando RUN a ver si vemos algo, lo único que podemos resaltar es cuando dicho salto toma este valor:

```
Address   Hex dump        Disassembly                    Comment
00427264  .  0375 00      add     esi,[ebp]
00427267  >  8A1E         mov     bl,[esi]
00427269  .  83EE FF      sub     esi,-1
0042726C  .  0FB6C3       movzx   eax,bl
0042726F  .- FF2485 00B04 jmp     [eax*4+46B000]         UnPackMe.0046B43F
00427276  .  0FB606       movzx   eax,byte ptr [esi]
00427279  .  66:8B4D 00   mov     cx,[ebp]
0042727D  .  83EE FF      sub     esi,-1
00427280  .  83C5 02      add     ebp,2
00427283  .  66:890C07    mov     [edi+eax],cx
00427287  .^ E9 DBFFFFFF  jmp     00427267
0042728C  .  80E0 3C      and     al,3C
0042728F  .  8B4D 00      mov     ecx,[ebp]
```

Si llegamos hasta allí vemos:

```
Address   Hex dump        Disassembly                    Comment
0046B43F    89EC          mov     esp,ebp
0046B441    58            pop     eax
0046B442    5B            pop     ebx
0046B443    5D            pop     ebp
0046B444    59            pop     ecx
0046B445    5E            pop     esi
0046B446    9D            popfd
0046B447    5F            pop     edi
0046B448    58            pop     eax
0046B449    5A            pop     edx
0046B44A    5F            pop     edi
0046B44B    C3            ret                            hummmmmmmm..
```

Esta zona ya me gusta más, sobretodo por ese POPFD y el RET posterior, si además nos fijamos en EDX tenemos una API ;)

```
Registers (FPU)
EAX 00000044
ECX 00000000
EDX 77E5D142 kernel32.GetVersion
EBX 0012FF44
ESP 0012FE5C
EBP 0012FF1C
ESI 0046CB16 UnPackMe.0046CB16
```

Esto no tendría más importancia de no ser que una vez superado el RET de :46B44B llegamos al inicio de ésta:

Address	Hex dump	Disassembly		Comment
77E5D142	64:A1 18000000	mov	eax,fs:[18]	GetVersion!
77E5D148	8B48 30	mov	ecx,[eax+30]	
77E5D14B	8B81 B0000000	mov	eax,[ecx+B0]	
77E5D151	0FB791 AC00000	movzx	edx,word ptr [ecx+AC]	
77E5D158	83F0 FE	xor	eax,FFFFFFFE	
77E5D15B	C1E0 0E	shl	eax,0E	
77E5D15E	0BC2	or	eax,edx	
77E5D160	C1E0 08	shl	eax,8	
77E5D163	0B81 A8000000	or	eax,[ecx+A8]	
77E5D169	C1E0 08	shl	eax,8	
77E5D16C	0B81 A4000000	or	eax,[ecx+A4]	
77E5D172	C3	ret		

Allí está, una vez ejecutada nos pone en una sección que no es la de código donde veníamos ejecutando código (en el salto indirecto), es aquí:

Address	Hex dump	Disassembly		Comment
0046CDB0	04 0C	add	al,0C	
0046CDB2	001438	add	[eax+edi],dl	
0046CDB5	3008	xor	[eax],cl	
0046CDB7	101C77	adc	[edi+esi*2],bl	
0046CDBA	68 F559BC96	push	96BC59F5	
0046CDBF	E8 97EEFFFF	call	0046BC5B	
0046CDC4	68 6577000D	push	0D007765	Retorna aqui!
0046CDC9	E8 43030000	call	0046D111	
0046CDCE	2A36	sub	dh,[esi]	
0046CDD0	26:0A02	or	al,es:[edx]	
0046CDD3	0E	push	cs	
0046CDD4	2E:1E	push	ds	Superfluous prefix
0046CDD6	1216	adc	dl,[esi]	
0046CDD8	06	push	es	

Allí vemos que corresponde a otra sección:

Address	Size	Owner	Section	Contains	Type	Access	Initial	Mapped as
003E0000	00003000				Priv	RW	RW	
003F0000	00002000				Map	R	R	
00400000	00001000	UnPackMe		PE header	Imag	R	RWE	
00401000	0004A000	UnPackMe	.text	code	Imag	R	RWE	
0044B000	0000C000	UnPackMe	.rdata		Imag	R	RWE	
00457000	00009000	UnPackMe	.data	data	Imag	R	RWE	
00460000	00003000	UnPackMe	.idata	imports	Imag	R	RWE	
00463000	00008000	UnPackMe	.rsrc	resources	Imag	R	RWE	
0046B000	00003000	UnPackMe	.vmp0		Imag	R	RWE	
0046E000	00001000	UnPackMe	.reloc	relocations	Imag	R	RWE	
00470000	00001000	ole32		PE header	Imag	R	RWE	
00471000	00104000	ole32	.text	code,import	Imag	R	RWE	
00575000	00006000	ole32	.orpc	code	Imag	R	RWE	

Así que hizo lo siguiente: Ejecutó código dentro de la sección CODE dentro de la rutina que contiene el salto indirecto, o sea, que en realidad actúa de loop, llega un punto en que en EDX aparece una API y el salto se desvía hacia :46B43F donde está el POPFD y el RET que nos deja en la API. Al pasar ésta nos deja en la sección .vmp0 y desde aquí continúa saltando nuevamente a la sección CODE (de código).

Por mucho que lo intentemos está metido en "su mundo" y de ahí no lo sacamos, se metió en ese loop y no lo suelta, si somos persistentes y seguimos dando RUN veremos que el salto vuelve a tomar ese mágico valor que le hace saltar hacia la dirección :046B43F como ya hizo antes:

Address	Hex dump	Disassembly		Comment
00427264	. 0375 00	add	esi,[ebp]	
00427267	> 8A1E	mov	bl,[esi]	
00427269	. 83EE FF	sub	esi,-1	
0042726C	. 0FB6C3	movzx	eax,bl	
0042726F	.- FF2485 00B04	jmp	[eax*4+46B000]	UnPackMe.0046B463
00427276	. 0FB606	movzx	eax,byte ptr [esi]	
00427279	. 66:8B4D 00	mov	cx,[ebp]	
0042727D	. 83EE FF	sub	esi,-1	
00427280	. 83C5 02	add	ebp,2	
00427283	. 66:890C07	mov	[edi+eax],cx	
00427287	.^ E9 DBFFFFFF	jmp	00427267	
0042728C	. 80E0 3C	and	al,3C	
0042728F	. 8B4D 00	mov	ecx,[ebp]	

Pero esta vez EDX no nos dice ABSOLUTAMENTE NADA ??

Registers (FPU)		
EAX	00000044	
ECX	00000000	
EDX	00000A28	hummmm...
EBX	00000A44	
ESP	0012FE54	
EBP	0012FF1C	
ESI	0046C714	UnPackMe.0046C714
EDI	0012FE54	

Bueno traceemos hasta el RET (de :46B44B) y lo pasamos..

```
Address   Hex dump     Disassembly                    Comment
004293A0  .  6A 00     push   0                       MaximumSize = 0
004293A2  .  68 00100000 push 1000                    InitialSize = 1000 (4096.)
004293A7  .  6A 00     push   0                       Flags = 0
004293A9  .  FF15 A009460 call [<&KERNEL32.HeapCreate> HeapCreate
004293AF  .  85C0      test   eax,eax
004293B1  .  A3 C4EB4500 mov  [45EBC4],eax
004293B6  .v 75 01     jnz    short 004293B9
004293B8  .  C3        ret
004293B9  >  E8 22000000 call 004293E0
004293BE  .  85C0      test   eax,eax
004293C0  .v 75 0F     jnz    short 004293D1
004293C2  .  A1 C4EB4500 mov  eax,[45EBC4]
004293C7  .  50        push   eax                     hHeap => NULL
004293C8  .  FF15 9C09460 call [<&KERNEL32.HeapDestroy HeapDestroy
004293CE  .  33C0      xor    eax,eax
004293D0  .  C3        ret
004293D1  >  B8 01000000 mov  eax,1
004293D6  .  C3        ret
```

Vaya por donde¡!! Esta vez tocó la sección CODE y PARA EJECUTAR CÓDIGO¡!! Pero está claro que esto NO es un OEP!

Investiguemos un poco, salgamos de aquí con CTRL+F9 (Execute till Return) y esto es lo que tenemos:

```
Address   Hex dump     Disassembly                    Comment
0046D26A  68 5817B4CA  push   CAB41758                Salimos aqui!!
0046D26F  E8 11E8FFFF  call   0046BA85
0046D274  68 F5CB4600  push   0046CBF5
0046D279  E8 C89FFBFF  call   00427246
0046D27E  68 A17225EB  push   EB2572A1
0046D283  E8 80F5FFFF  call   0046C808
0046D288  1A0A         sbb    cl,[edx]
0046D28A  36:0206      add    al,ss:[esi]
0046D28D  16           push   ss
```

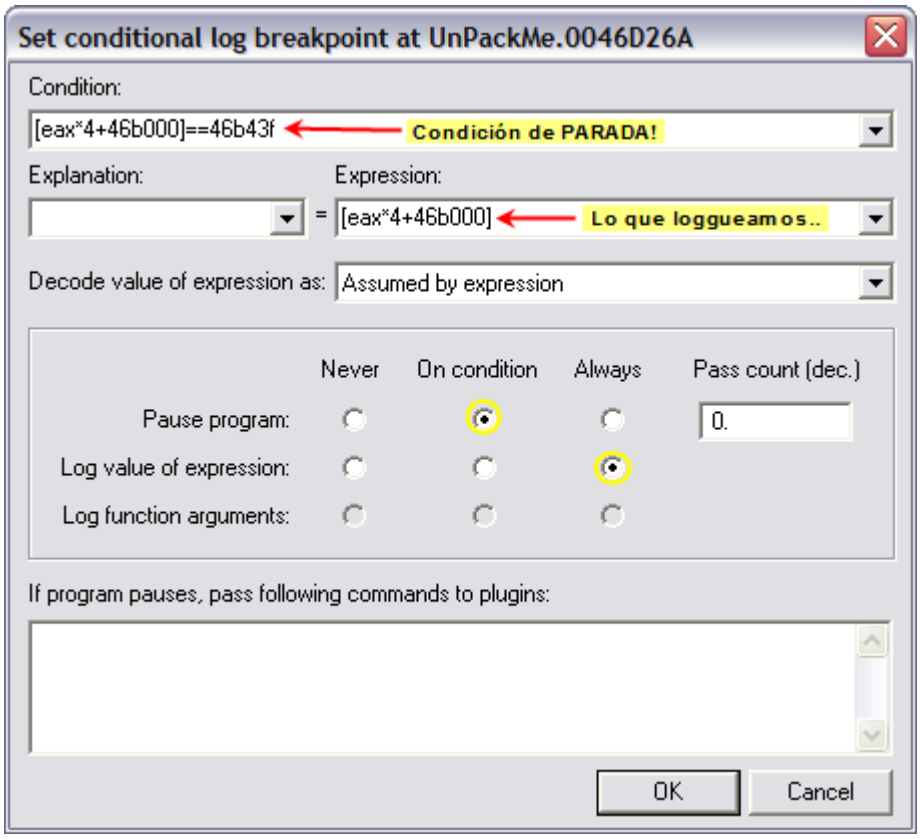
Hummm.. vuelve a la sección .vmp0, bueno parece que vamos obteniendo un patrón de conducta, es evidente que a estas alturas el OEP ya está superado, sino abramos el UnPackMe de Teddy Roggers (sin empakar) y coloquemos un BP en :4293A0, ejecutemos y al parar pulsemos CTRL+F9 para retornar, esto es lo que vemos:

```
Address   Hex dump     Disassembly                    Comment
004271B0  55           push   ebp
004271B1  8BEC         mov    ebp,esp
004271B3  6A FF        push   -1
004271B5  68 600E4500  push   00450E60
004271BA  68 C8924200  push   004292C8                SE handler installation
004271BF  64:A1 00000000 mov  eax,fs:[0]
004271C5  50           push   eax
004271C6  64:8925 000000 mov  fs:[0],esp
004271CD  83C4 A8      add    esp,-58
004271D0  53           push   ebx
004271D1  56           push   esi
004271D2  57           push   edi
004271D3  8965 E8      mov    [ebp-18],esp
004271D6  FF15 DC0A4600 call  [<&kernel32.GetVersion>] kernel32.GetVersion
004271DC  33D2         xor    edx,edx
004271DE  8AD4         mov    dl,ah
004271E0  8915 34E64500 mov   [45E634],edx
004271E6  8BC8         mov    ecx,eax
004271E8  81E1 FF000000 and   ecx,0FF
004271EE  890D 30E64500 mov   [45E630],ecx
004271F4  C1E1 08      shl    ecx,8
004271F7  03CA         add    ecx,edx
004271F9  890D 2CE64500 mov   [45E62C],ecx
004271FF  C1E8 10      shr    eax,10
00427202  A3 28E64500  mov    [45E628],eax
00427207  E8 94210000  call   004293A0
0042720C  85C0         test   eax,eax                Salimos aqui!
0042720E  v 75 0A      jnz    short 0042721A
```

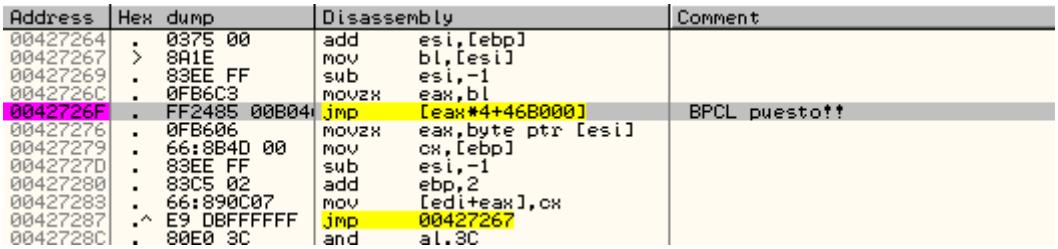
Dentro del call de :42720C está el código al que hemos llegado, está claro que esto tiene muchísimos Stolen Bytes, pues el código faltante lo debió de ejecutar en el propio packer y después saltar aquí directamente. De esto deducimos que cada vez que pisa la sección .vmp0 es para ejecutar una llamada (call) dentro del EXE protegido.

Está claro lo que hace, podríamos pues meterle un BrekPoint tipo Condicional en el salto indirecto para que se detenga cuando toma el valor que nos envía a la dirección mágica :D

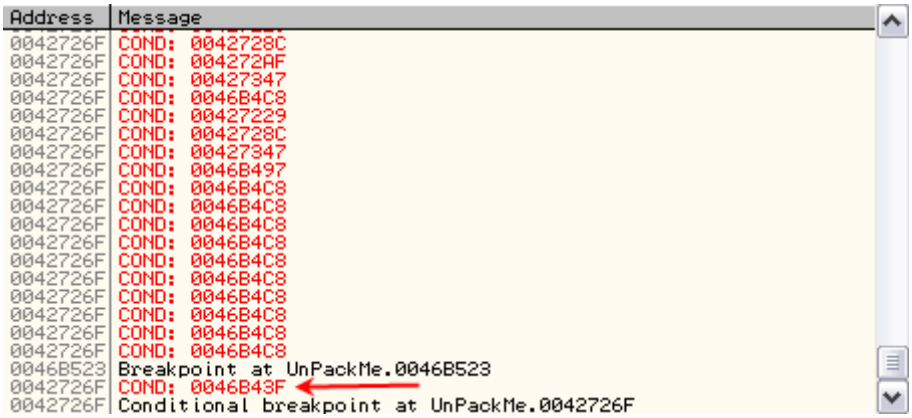
Pulsamos SHIFT+F4 y rellenamos el BP con estos datos:



Antes reiniciamos Olly con CTRL+F2 y vamos a la ventana Log con ALT+L, allí limpiamos TODO con el botón derecho del ratón y seleccionando CLEAR WINDOW, el salto debe quedar así:



acto seguido damos RUN y al poco Olly para, miremos la ventana Log con ALT+L a ver que vemos:



Allí paró por la que nosotros queremos, a estas alturas sabemos que el OEP ya lo dejamos atrás, pero eso ahora que no nos despiste, está a punto de saltar a la zona del POPFD-RET que nos lleva a GetVersion, pero investiguemos un poco antes, sigamos con F9 desde aquím y cuando pare otra vez estamos en la situación anterior, pero démosle RUN nuevamente a ver donde nos lleva:

```
Address   Hex dump     Disassembly                          Comment
0042A1EE     90        nop
0042A1EF     90        nop
0042A1F0  .  56        push    esi                           3a parada!
0042A1F1  .  E8 BAE5FFFF call    004287B0
0042A1F6  .  FF15 500A460( call  [<&KERNEL32.TlsAlloc>      TlsAlloc
0042A1FC  .  83F8 FF     cmp     eax,-1
0042A1FF  .  A3 B4B54500 mov     [45B5B4],eax
0042A204  .v 74 42       je      short 0042A248
0042A206  .  6A 74       push    74
0042A208  .  6A 01       push    1
0042A20A  .  E8 71A3FFFF call    00424580
0042A20F  .  8BF0        mov     esi,eax
0042A211  .  83C4 08     add     esp,8
0042A214  .  85F6        test    esi,esi
0042A216  .v 74 30       je      short 0042A248
0042A218  .  A1 B4B54500 mov     eax,[45B5B4]
0042A21D  .  56          push    esi                          pValue
0042A21E  .  50          push    eax                          TlsIndex => FFFFFFFF
0042A21F  .  FF15 380A460( call  [<&KERNEL32.TlsSetVal       TlsSetValue
0042A225  .  85C0        test    eax,eax
0042A227  .v 74 1F       je      short 0042A248
0042A229  .  56          push    esi
0042A22A  .  E8 21000000 call    0042A250
0042A22F  .  83C4 04     add     esp,4
0042A232  .  FF15 CC0A460( call  [<&KERNEL32.GetCurren       GetCurrentThreadId
0042A238  .  8906        mov     [esi],eax
0042A23A  .  C746 04 FFFF( mov   dword ptr [esi+4],-1
0042A241  .  B8 01000000 mov     eax,1
0042A246  .  5E          pop     esi
0042A247  .  C3          ret
```

Ahí está, justo en el EXE, esto lo hacemos para asegurarnos que vamos bien, pero también podríamos perfectamente NO hacerlo. Si seguimos apretando F9 iremos cayendo en diferentes zonas del EXE, lo que nos confirma lo que veníamos pensando hasta ahora.

Podemos pegarnos horas así, por lo que ya lo dejamos, la idea era esa (la de ver lo que hace). Lo siguiente más que descubir el OEP que será complicado será escoger un OEP, que quiero decir, que en estos casos en los que va saltando de la sección de código a la sección del packer lo que se puede hacer es elegir un punto del EXE donde pare la primera vez (lo que sería un OEP) y dumpear desde allí. Pero si hacemos esto así sin más NO funcionará por el tema de la pila que habrá que equilibrarla después, de manera que quede como en el empaquetado en ese punto. Esto no es difícil de conseguir ni mucho menos así que vamos a ello ;)

-** Eligiendo un OEP **-

Pues si, esta vez se lo vamos a poner nosotros (el que queramos) siempre que esté dentro del EXE claro está, pues sabemos que una vez superado éste el programa continúa saltando a la sección del packer, digamos que está entremezclado (tipo EXECryptor) pero sin ser tan complejo ni muchísimo menos.

La idea pues es la de colocar un BPM en la sección de código y una vez pare allí dumpear, pero tenemos el problema que está en un loop dentro de dicha sección donde se encuentra el salto indirecto.

Entonces lo que hacemos es reiniciar Olly con CTRL+F" y ctivar el Breakpoint Condicional Log que pusimos antes. Cuando pare la primera vez sabemos que saltará a GetVersion por lo que seguimos con F9 y parará aquí:

```
Address   Hex dump   Disassembly                          Comment
004293A0  .  6A 00        push    0                      MaximumSize = 0
004293A2  .  68 00100000  push    1000                   InitialSize = 1000 (4096.)
004293A7  .  6A 00        push    0                      Flags = 0
004293A9  .  FF15 A009460 call    [<&KERNEL32.HeapCreate> HeapCreate
004293AF  .  85C0         test    eax,eax
004293B1  .  A3 C4EB4500  mov     [45EBC4],eax
004293B6  .v 75 01        jnz     short 004293B9
004293B8  .  C3           ret
004293B9  >  E8 22000000  call    004293E0
004293BE  .  85C0         test    eax,eax
004293C0  .v 75 0F        jnz     short 004293D1
004293C2  .  A1 C4EB4500  mov     eax,[45EBC4]
004293C7  .  50           push    eax                    hHeap => NULL
004293C8  .  FF15 9C09460 call    [<&KERNEL32.HeapDestroy HeapDestroy
004293CE  .  33C0         xor     eax,eax
004293D0  .  C3           ret
004293D1  >  B8 01000000  mov     eax,1
004293D6  .  C3           ret
```

Ese podría ser perfectamente nuestro OEP, dumpeamos con OllyDump destildando en "Rebuild Import" que repararemos luego con el ImpREC, si miramos la tabla vemos que es correcta:

```
Address   Hex dump                                         ASCII
00460818  D8 17 DA 77 9A 22 DA 77 D6 27 DA 77 B1 63 DB 77  ï‡ rwÜ"rwï' rw▒c▄w
00460828  10 24 DA 77 00 00 00 00 B1 38 31 77 3D 51 31 77  ►$ rw....▒81w=Q1w
00460838  00 00 00 00 02 3D C4 77 00 42 C4 77 45 29 C4 77  ....☻=─w.B─wE)─w
00460848  9D 50 C4 77 10 85 C4 77 7E 4F C4 77 BC 6C C4 77  ØP─w►à─w~O─w╝l─w
00460858  50 33 C4 77 E1 32 C4 77 0B ED C6 77 B2 92 C6 77  P3─wß2─w♂ÿšw▒æšw
00460868  06 8A C4 77 DC 7E C4 77 2F 52 C4 77 45 92 C4 77  ♠è─w▄~─w/R─wEæ─w
00460878  80 70 C4 77 75 8D C4 77 FA 99 C4 77 A2 43 C5 77  Çp─wuì─w·Ö─wóC┼w
00460888  C7 5A C4 77 90 84 C4 77 8E 21 C4 77 EE 63 C4 77  ÃZ─wÉä─wÄ!─w^c─w
00460898  CE 97 C4 77 D1 36 C4 77 54 93 C4 77 22 4F C4 77  ╬ù─wÐ6─wTô─w"O─w
004608A8  18 B6 C4 77 76 80 C4 77 A9 2B C4 77 A2 42 C5 77  ↑Á─wvÇ─w®+─wóB┼w
004608B8  12 95 C6 77 6E 3C C4 77 B4 DC C6 77 9D 88 C4 77  ↕òšwn<─w┤▄šwØê─w
004608C8  C6 83 C4 77 E0 20 C4 77 98 1B C4 77 68 4E C4 77  ãä─wó ─wÿ←─whN─w
004608D8  F8 A9 C4 77 3C 65 C4 77 C0 C8 C5 77 F9 AA C4 77  °®─w<e─w└╚┼w··¬─w
```

Otra cosa importante es como dije antes LA PILA, veámosla en el OEP:

```
Address   Value    Comment
0012FF48  0046D26A UnPackMe.0046D26A
0012FF4C  0000000F
0012FF50  0012CD2C
0012FF54  7FFDF000
0012FF58  EE897C8C
0012FF5C  804EC226
0012FF60  EE897C7C
0012FF64  0000001E
0012FF68  8053C88F
0012FF6C  EE897CFC
0012FF70  F27D2A44
0012FF74  00000000
0012FF78  81CBF008
0012FF7C  804E8996
0012FF80  823CEA00
0012FF84  00000000
0012FF88  00000001
0012FF8C  00000000
0012FF90  0046CAC5 RETURN to UnPackMe.0046CAC5 from UnPackMe.00427246
0012FF94  00000000
0012FF98  7FFDF000
```

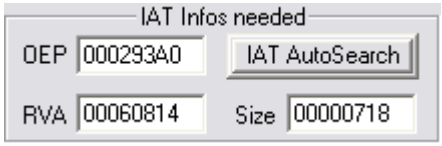
Es importante este valor para después injertar¡!!!!!

Ese es el valor que tiene en el OEP elegido, recordemos ese 0x12FF48 para después, vamos al mapa de memoria ALT+M y picamos sobre la sección de PILA, en mi caso es esta:

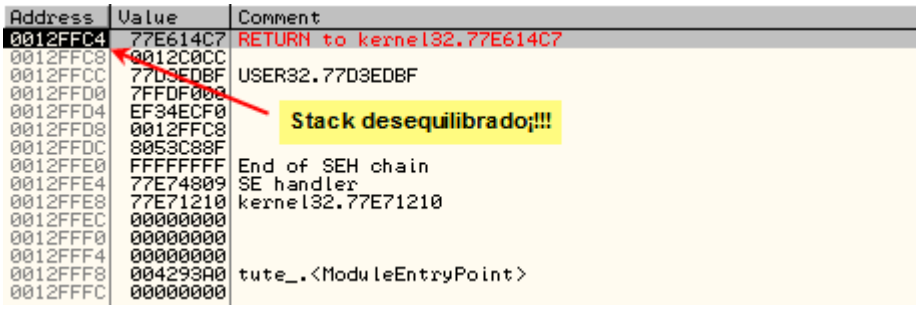
```
Address   Size     Owner  Section  Contains   Type Access  Initial Map
00010000  00001000                            Priv RW      RW
00020000  00001000                            Priv RW      RW
0012C000  00001000                            Priv RW  Gua RW
0012D000  00003000                stack of ma Priv RW  Gua RW
00130000  00001000                            Map  R       R
00140000  00026000                            Priv RW      RW
00240000  00006000                            Priv RW      RW
00250000  00001000                            Map  RW      RW
00260000  00016000                            Map  R       R       \De
00280000  00034000                            Map  R       R       \De
002C0000  00041000                            Map  R       R       \De
00310000  00006000                            Map  R       R       \De
```

Al lado además lo pone, picamos con el botón derecho y DUMP IN CPU para ver los bytes en el DUMP, seleccionamos el primer byte y pinchamos con el botón izquierdo sobre él, después con el derecho y elegimos COPY-SELECT ALL, después BINARY COPY. Estos bytes es importante NO perderlos, es incluso recomendable guardarlos con un editor hexadecimal en un archivo.

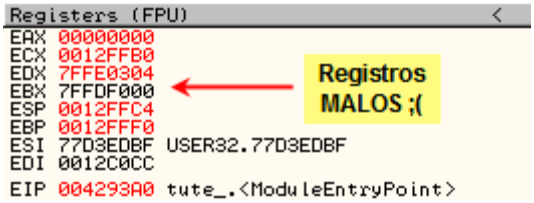
En cualquier caso abrimos el ImpREC y le damos los datos correctos, recordando que el OEP lo hemos puesto en 0x4293A0, o sea que pondremos:



Luego pulsamos GET IMPORTS + SHOW INVALID (todas bien) + FIX DUMP, elegimos tute.exe y nos dejará tute_.exe que todavía NO funciona. Si cargamos el dumpeado tute_.exe en otro Olly veremos porque:



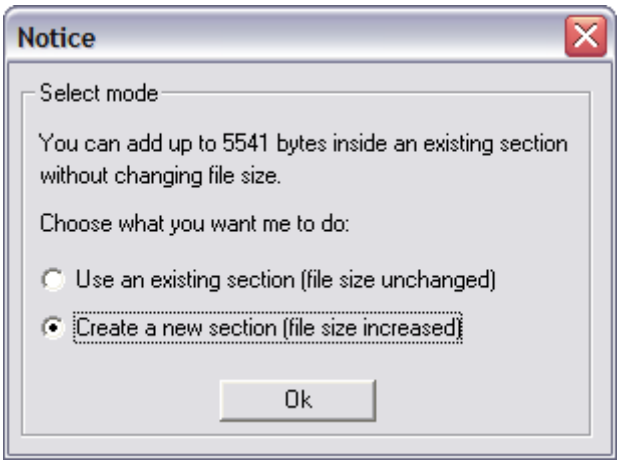
Ahí lo vemos, además los registros TAMPOCO son correctos:



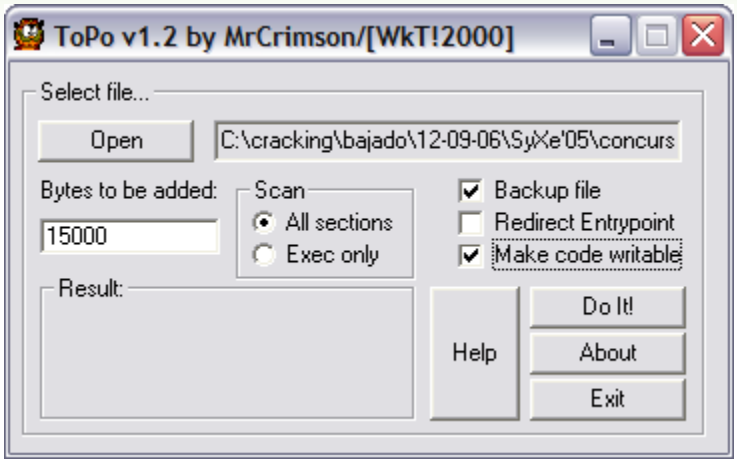
Lo que hacemos es injertar la pila en el dumpeado y modificar los registros, para después dejarlo correr desde el OEP.

Lo primero de todo es abrir ToPo v1.2 y agregar una nueva sección a tute_.exe de al menos 3000h bytes como tiene la pila, además de dejarle espacio para escribir el inejrtito que haremos.

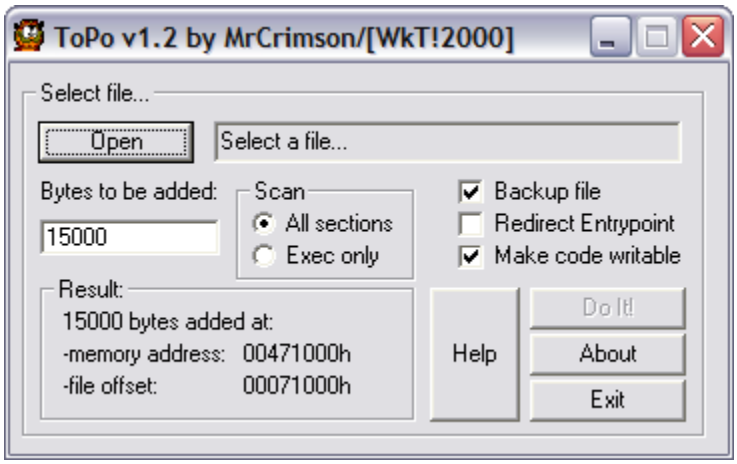
Como hay que pasar el 0x3000h a decimal serían 12288d. así que le agregamos como 15000d. bytes para estar tranquilos



De momento le decimos que nos cree una nueva sección en el ejecutable pues los 5541 bytes que hay libres no nos sirven (además no me gusta hacerlo así =)



Ahí le ponemos los 1555d bytes a agregar, ojo con la opción MAKE CODE WRITABLE que debe estar activada para después poder escribir código o lo que queramos sobre ella, le damos a DO IT!! Y nos hace el trabajo:



Ahí tenemos nuestro espacio reservado, a partir de :471000, abrimos un OllyDbg y cargamos el dumpeado tute_.exe al que le acabamos de agregar una sección, vamos al mapa de memoria y la vemos:

Address	Size	Owner	Section	Contains	Type	Access	Initial	Mapped
003E0000	00001000				Priv	RWE	RWE	
003F0000	00001000				Priv	RWE	RWE	
00400000	00001000	tute_		PE header	Imag	R	RWE	
00401000	0004A000	tute_	.text	code	Imag	R	RWE	
0044B000	0000C000	tute_	.rdata		Imag	R	RWE	
00457000	00009000	tute_	.data	data	Imag	R	RWE	
00460000	00003000	tute_	.idata		Imag	R	RWE	
00463000	00008000	tute_	.rsrc	resources	Imag	R	RWE	
0046B000	00003000	tute_	.vmp0		Imag	R	RWE	
0046E000	00001000	tute_	.reloc	relocations	Imag	R	RWE	
0046F000	00002000	tute_	.mackt	imports	Imag	R	RWE	
00471000	00004000	tute_	.topo0		Imag	R	RWE	
00480000	00009000				Map	R E	R E	

Y con el tamaño 4000h bytes, perfecta para albergar los bytes de pila y el injerto, botón derecho sobre ella y DUMP IN CPU, picamos sobre el primer btye con el botón izquierdo, luego con el derecho y elegimos COPY-SELECT ALL para que los seleccione TODOS, es hora de ir al Olly donde tenemos el crackme parados en el OEP, o bien al archivo binario o donde hayamos puesto los bytes de la zona de pila 0x3000h bytes, hacemos un BINARY COPY y nos vamos a la zona injertada y hacemos un BINARY PASTE, deberán quedar algo así:

Address	Hex dump	ASCII
00471000	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00471010	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00471020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00471030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00471040	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00471050	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00471060	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00471070	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00471080	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00471090	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
004710A0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
004710B0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
004710C0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
004710D0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
004710E0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
004710F0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00471100	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....

Allí están TODOS guardaditos, para el injerto sólo es necesario saber 2 cosas:

- Dirección donde se guardan (0x471000h)
- Número de bytes (0x3000h)

Esto lo tenemos todo, vamos para abajo hasta encontrar un sitio para injertar, donde pondremos el código necesario:

Address	Hex dump	ASCII
00473F80	00 EA 3C 82 00 00 00 00 01 00 00 00 00 00 00 00	.Ü<é....☺.....
00473F90	C5 CA 46 00 00 00 00 00 00 F0 FD 7F F0 FF 12 00	┼╩F.....-²▌- ↕.
00473FA0	B0 FF 12 00 12 02 00 00 4C FF 12 00 58 FF 12 00	░ ↕.↕☻..L ↕.X ↕.
00473FB0	E0 FF 12 00 C8 92 42 00 60 0E E5 00 FF FF FF FF	ó ↕.╚æB.`♫E.
00473FC0	F0 FF 12 00 C7 14 E6 77 00 00 00 00 00 00 00 00	- ↕.╟¶µw.....
00473FD0	00 F0 FD 7F F0 AC 90 EE C8 FF 12 00 8F C8 53 80	.-²▌-¼É-╚ ↕.Ä╚SÇ
00473FE0	FF FF FF FF 09 48 E7 77 10 12 E7 77 00 00 00 00	....H╫w►↕╫w....
00473FF0	00 00 00 00 00 00 00 00 B0 71 42 00 00 00 00 00	.....☼qB.....
00474000	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	ÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉ
00474010	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	ÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉ
00474020	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	ÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉ
00474030	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	ÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉ
00474040	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	ÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉ
00474050	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	ÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉ

Ahí justo después de los bytes salvados comienza una serie de 0x90s que nos vienen de perlas para el asunto, vamos allí y ponemos esto:

Address	Hex dump	Disassembly		Comment
00474000	90	nop		
00474001	BE 00104700	mov	esi,00471000	
00474006	BF 00D01200	mov	edi,12D000	
0047400B	B9 00300000	mov	ecx,3000	
00474010	F3:A4	rep	movsb	
00474012	90	nop		
00474013	B8 280A0000	mov	eax,0A28	
00474018	B9 01050000	mov	ecx,501	
0047401D	BA 01000000	mov	edx,1	
00474022	BB 00F0FD7F	mov	ebx,7FFDF000	
00474027	BC 48FF1200	mov	esp,12FF48	
0047402C	BD C0FF1200	mov	ebp,12FFC0	
00474031	BE 8B57F477	mov	esi,77F4578B	
00474036	BF 60CD1200	mov	edi,12CD60	
0047403B	90	nop		
0047403C	68 A0934200	push	<ModuleEntryPoint>	-> push 4293A0
00474041	C3	ret		
00474042	90	nop		
00474043	90	nop		
00474044	90	nop		
00474045	90	nop		
00474046	90	nop		
00474047	90	nop		
00474048	90	nop		
00474049	90	nop		
0047404A	90	nop		
0047404B	90	nop		

Es muy sencillo ESI debe apuntar a la zona que actuará de BUFFER en la copia de datos (bytes a copiar), EDI como puntero a la zona donde se copiarán (Zona de pila) y ECX el número de bytes a copiar, luego el REP MOVS hace el trabajo, actualizamos los registros y saltamos al OEP con un PUSH-RET. Sencillo =)

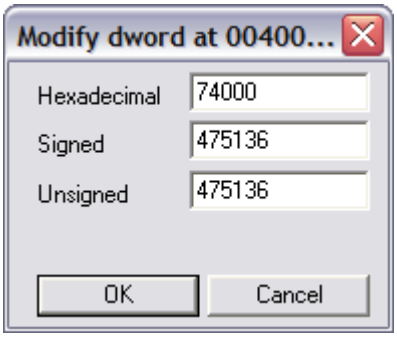
Seleccionamos estos bytes junto con los que formarán parte de la pila y que acabamos de agregar (todo en la última sección .topo) picamos con el botón derecho y seleccionamos COPY TO EXECUTABLE FILE + SAVE FILE en la siguiente pantalla que nos dejará.

Sólo nos queda probar el desempakado, le hacemos doble clic y.. NO funciona, grrrrr.. claro el OEP¡!!!

Debemos desviar el OEP hacia el injerto, sino arrancará desde :4293A0 y claro no funciona, jeje.. lo cambiamos desde el propio Olly, lo abrimos y vamos al PE header, lo ponemos en modo SPECIAL PE HEADER y lo buscamos:

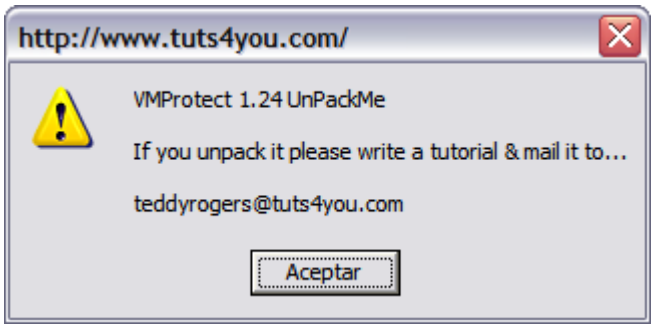
```
00400094   E000       DW 00E0          SizeOfOptionalHeader = E0 (224.)
00400096   0E01       DW 010E          Characteristics = EXECUTABLE_IMAGE|32B
00400098   0B01       DW 010B          MagicNumber = PE32
0040009A   05         DB 05            MajorLinkerVersion = 5
0040009B   00         DB 00            MinorLinkerVersion = 0
0040009C   00920400   DD 00049200      SizeOfCode = 49200 (299520.)
004000A0   007A0100   DD 00017A00      SizeOfInitializedData = 17A00 (96768.)
004000A4   00000000   DD 00000000      SizeOfUninitializedData = 0
004000A8   A0930200   DD 000293A0      AddressOfEntryPoint = 293A0
004000AC   00100000   DD 00001000      BaseOfCode = 1000
004000B0   00B00400   DD 0004B000      BaseOfData = 4B000
004000B4   00004000   DD 00400000      ImageBase = 400000
004000B8   00100000   DD 00001000      SectionAlignment = 1000
004000BC   00020000   DD 00000200      FileAlignment = 200
```

Ahí lo tenemos, queremos que arranque desde :474000 así que picamos sobre él con el botón derecho y elegimos MODIFY INTEGER, luego lo cambiamos:



Aceptamos en OK y luego seleccionamos los bytes en el DUMP (dirección 0x4000A8h) y otra vez el COPY TO EXECUTABLE FILE + SAVE FILE.

Y ahora no debería dar problemas, lo ejecutamos y..



jejejeje.. UnPackMe rulandoooooooooo =))

Podemos comprobar como pasado el OEP todavía sigue saltando al packer, por lo que la opción del injerto pasa a ser bajo mi punto de vista de las mejores :D

-** Saludos y/o Agradecimientos **-

Y bueno, ya lo saben ELLOS, Todos lo de la lista CracksLatinoS y la banda el yahoo que nos juntamos cada día X)

A los autores de todas las tools aquí utilizadas:

- Oleh Yuschuk por OllyDbg 1.10
- MackT por Import Reconstructor v1.6f
- rdgmax por RDG Packer Detector v0.6.5 Beta
- etc…

Y bueno, poco más que decir de un UnPackMe un tanto EXTRAÑO, no sé un desempakado un tanto atípico y que no me gusta mucho pero bueno, jeje.. ya vendrán mejores si puede ser ;)

-** Contacto **-

El autor puede ser contactado mediante esta dirección de correo electrónico:

syxe05@gmail.com